

Academia Portal: Smart Course Scheduling System

Operating Systems Mini Project Report

Vishudha Sood

Roll Number: IMT2024067

IIIT Bangalore

April 30, 2026

Abstract

The Academia Portal is a role-based academic management system developed in C using Linux system programming principles. It simulates a university portal enabling Admins, Faculty, and Students to securely interact through a multi-threaded client-server architecture. The project integrates core Operating Systems concepts including socket programming, file locking, synchronization, inter-process communication, persistent storage, and concurrency. Key functionalities include user authentication, course creation, enrollment management, waitlist handling, drop functionality, and automated waitlist promotion. This project demonstrates practical implementation of OS principles while delivering a scalable and resume-worthy academic solution.

Contents

1	Introduction	3
2	Project Objectives	3
3	System Architecture	4
3.1	Components	4
3.2	Major Files	4
4	Implementation Details	4
4.1	Step 1: Basic Client-Server Communication	4
4.2	Step 2: Authentication System	5
4.3	Step 3: Menu-Based Dashboard	6
4.4	Step 4: Persistent Data Management	6
4.5	Step 5: Enrollment and Waitlist System	7

5	Advanced Operating Systems Concepts Used	7
5.1	File Locking	7
5.2	Concurrency	8
5.3	Synchronization	8
5.4	Inter-Process Communication	8
6	Key Features	9
7	Sample Workflow	9
7.1	Admin Workflow	9
7.2	Faculty Workflow	9
7.3	Student Workflow	10
8	Project Strengths	10
9	Limitations	10
10	Future Enhancements	10
11	Conclusion	11

1 Introduction

The **Academia Portal** is a Linux-based client-server academic management system developed in C as part of an Operating Systems mini-project. The project simulates a real-world university portal where different stakeholders—**Admin, Faculty, and Students**—can interact through secure role-based access.

The system was designed to incorporate key Operating Systems concepts such as:

- Socket Programming
- Client-Server Architecture
- File Handling and Persistent Storage
- File Locking using `fcntl()`
- Multi-threading using POSIX Threads
- Synchronization using Mutexes
- Inter-Process Communication (FIFO + Signals)
- Logging and Resource Management

The portal provides practical features such as:

- User Authentication
- Student/Faculty Management
- Course Creation
- Course Enrollment
- Waitlist Handling
- Auto-Promotion from Waitlist
- Drop Course Functionality

This project was developed with the objective of creating a modular, practical, and resume-worthy system while strongly aligning with core Operating Systems principles.

2 Project Objectives

The primary objectives of this project are:

1. Build a robust academic management system that simulates real-world university operations.
2. Apply major Operating Systems concepts such as process management, networking, synchronization, and resource allocation in a practical project.
3. Ensure secure and role-based access to different system functionalities.

4. Develop persistent storage mechanisms using structured file handling.
5. Implement concurrency and synchronization to safely support multiple simultaneous users.
6. Create a scalable and demonstrable mini-project suitable for academic evaluation and professional portfolios.

3 System Architecture

The system follows a **multi-threaded client-server architecture**, where multiple clients interact with a centralized server.

3.1 Components

- **Client Module:** Provides terminal-based user interaction and communicates requests to the server.
- **Server Module:** Handles authentication, request processing, academic operations, and synchronization.
- **Database Layer:** Uses structured text files to store persistent academic records.
- **Utility Layer:** Implements logging, file locking, synchronization, and IPC.

3.2 Major Files

File	Purpose
server.c	Main server logic, socket handling, request processing
client.c	Client-side communication and user menus
students.txt	Stores student credentials
faculties.txt	Stores faculty credentials
courses.txt	Stores course records, capacities, and faculty assignments
enrollments.txt	Tracks student course enrollments
waitlist.txt	Maintains waitlisted students
logs.txt	Stores system event logs
Makefile	Automates compilation and build process

4 Implementation Details

4.1 Step 1: Basic Client-Server Communication

The first stage focused on establishing a reliable communication framework using TCP socket programming.

A dedicated server socket was created using `socket()`, which initialized a communication endpoint. The server then used `bind()` to associate itself with a specific IP and port. Through `listen()`, the server entered passive mode and continuously waited for client requests.

On the client side, a separate socket was created and connected using `connect()`. The server accepted incoming requests using `accept()`, which generated a new dedicated communication socket for that client.

This stage established:

- Reliable two-way communication
- Persistent server availability
- Foundation for all future role-based interactions
- Practical networking implementation

System Calls Used:

- `socket()`
- `bind()`
- `listen()`
- `accept()`
- `connect()`

4.2 Step 2: Authentication System

This phase introduced secure role-based authentication.

Three user categories were implemented:

- Admin with hardcoded superuser credentials
- Faculty validated via `faculties.txt`
- Students validated via `students.txt`

The login process involves:

- Role selection
- Username verification
- Password verification

This ensures:

- Secure user access
- Role segregation
- Authorization boundaries
- Controlled privilege distribution

4.3 Step 3: Menu-Based Dashboard

After successful authentication, users are redirected to dedicated dashboards.

Admin Dashboard: Enables centralized academic administration.

Faculty Dashboard: Allows course management.

Student Dashboard: Provides course browsing and enrollment access.

Persistent menu loops ensure:

- Session continuity
- Dynamic server-client interactions
- Repeated operations
- Secure logout

This transformed the system into an interactive management platform.

4.4 Step 4: Persistent Data Management

This phase introduced real-world persistence through structured text files.

Admin Features:

- Add students to the academic system
- Add faculty members
- View all registered users

Faculty Features:

- Add courses
- Assign capacities
- View all courses

Student Features:

- Browse available courses
- Monitor faculty assignments
- Check seat capacities

This stage introduced:

- Persistent storage
- Structured file I/O
- Database simulation
- Administrative realism

4.5 Step 5: Enrollment and Waitlist System

This stage significantly enhanced system complexity.

Enrollment: Students may enroll in courses if seats are available.

Capacity Management: Each course stores:

- Total capacity
- Current enrollment

Waitlist System: If capacity is full:

- Student is automatically waitlisted
- Queue order is maintained
- Resource fairness is preserved

Drop Functionality: Students can drop courses.

Auto-Promotion: When seats open:

- First waitlisted student is promoted
- Enrollment files are updated
- Notifications/logs are triggered

This demonstrates:

- Scheduling algorithms
- Queue systems
- Resource allocation
- Transaction consistency

5 Advanced Operating Systems Concepts Used

5.1 File Locking

File locking prevents race conditions and inconsistent file modifications.

Implemented using:

```
fcntl(fd, F_SETLK, &fl);
```

This ensures:

- Safe concurrent access
- Controlled writes
- Data integrity
- Resource protection

5.2 Concurrency

Concurrency was introduced through POSIX threads:

```
pthread_create()
```

Capabilities include:

- Multiple simultaneous users
- Thread-per-client design
- Efficient resource utilization

5.3 Synchronization

Mutexes are used to synchronize:

- Log writes
- Shared files
- IPC communication

This prevents:

- Race conditions
- Deadlocks
- Data corruption

5.4 Inter-Process Communication

The system incorporates:

- Named Pipes (FIFO)
- Signals (SIGUSR1)

These mechanisms support:

- Notifications
- Waitlist updates
- Asynchronous communication

6 Key Features

- Secure role-based authentication
- Admin user management
- Faculty course creation
- Student course browsing
- Enrollment with seat limits
- Waitlist system
- Drop functionality
- Auto waitlist promotion
- Persistent storage
- Detailed logging
- File locking
- Concurrency support

7 Sample Workflow

7.1 Admin Workflow

1. The Admin logs into the system using secure administrative credentials.
2. After authentication, the Admin accesses a dashboard with full system privileges.
3. The Admin can register new students by securely adding credentials to the student database.
4. The Admin can similarly register faculty members.
5. The Admin may view all registered users to monitor institutional records.
6. Logging out securely terminates the administrative session.

7.2 Faculty Workflow

1. Faculty members login using validated credentials.
2. Upon successful login, they gain access to faculty-specific course management tools.
3. Faculty can create courses, assign capacities, and define academic offerings.
4. Faculty may monitor and review course records.
5. Logout preserves all academic data while ending the session.

7.3 Student Workflow

1. Students login using secure student credentials.
2. Students browse available courses along with seat capacities and faculty assignments.
3. Students may enroll directly if seats are available.
4. If a course is full, the student is automatically placed in a waitlist queue.
5. Students can view personal enrollments for planning purposes.
6. Students may drop courses, allowing dynamic seat reallocation.
7. Waitlisted students are automatically promoted when seats become available.
8. Logging out preserves all enrollment data securely.

8 Project Strengths

- Strong Operating Systems alignment
- Practical real-world application
- Modular architecture
- Resume-worthy implementation
- Networking + synchronization integration
- Resource management realism

9 Limitations

- File-based database instead of SQL
- Terminal-based interface
- Limited UI sophistication
- Basic security model
- Moderate scalability

10 Future Enhancements

- SQL/NoSQL database integration
- Web-based frontend
- Encryption and hashing
- Advanced scheduling algorithms

- Notification dashboards
- Analytics modules

11 Conclusion

The Academia Portal successfully demonstrates the integration of critical Operating Systems concepts into a practical academic management platform.

Through implementation of:

- Networking
- File systems
- Synchronization
- Multi-threading
- IPC
- Resource allocation

the project fulfills academic objectives while also providing strong professional relevance.

This system represents a comprehensive and practical demonstration of Linux system programming principles, making it highly suitable for both academic submission and portfolio enhancement.